



MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 1 2025/2026

**WEEK 9A & 9B: IMAGE/VIDEO INPUT INTERFACING WITH
MICROCONTROLLER (VER. 2A)**

SECTION 1

GROUP 9

**LECTURER: ZULKIFLI BIN ZAINAL ABIDIN & WAHJU
SEDIONO**

NO.	GROUP MEMBERS	MATRIC NO.
1.	AHMAD FIRDAUS HUSAINI BIN HASAN AL-BANNA	2315377
2.	ADIB ZAFFRY BIN MOHD ABDUL RADZI	2315149
3.	UMAR FAROUQI BIN ABU HISHAM	2314995

Date of Submission: 17 December 2025

ABSTRACT

This experiment focuses on the implementation of a vision based color detection system using an Arduino Uno and the Gravity HuskyLens AI Vision Camera. Although the original experiment plan included color detection using a conventional color sensor, this part could not be carried out due to the unavailability of the required hardware. Consequently, the experiment proceeded using the HuskyLens AI Camera, which performs onboard color recognition through image processing. The system was trained to recognize multiple colored objects, each assigned a unique identification number. An RGB LED module was integrated as a visual output indicator, where different LED colors were activated based on the detected color ID. The results demonstrate reliable real time color detection with good consistency under varying lighting conditions. This experiment highlights the effectiveness of AI assisted vision systems for color detection applications in modern mechatronics and embedded systems.

TABLE OF CONTENTS

NO	TOPIC	PAGE
1	INTRODUCTION	4
2	MATERIALS AND EQUIPMENTS	5
3	EXPERIMENTAL SETUP	6-7
4	METHODOLOGY	8-14
5	DATA COLLECTION	15
6	DATA ANALYSIS	16
7	RESULTS	17
8	DISCUSSION	18-19
9	CONCLUSION	20
10	RECOMMENDATIONS	21
11	REFERENCES	22
12	APPENDICES	23
13	ACKNOWLEDGEMENT	24-25
14	CERTIFICATE OF AUTHENTICITY	24-25

1.0 INTRODUCTION

Color detection is a fundamental capability in many mechatronics and automation systems, with applications in object sorting, quality inspection, robotics, and human machine interaction. Traditionally, color detection is implemented using dedicated color sensors that measure reflected light intensity in red, green, and blue channels. While such sensors are effective for simple tasks, their performance is often sensitive to environmental factors such as lighting conditions and surface reflections.

In this experiment, the focus was placed on vision based color detection using the Gravity HuskyLens AI Vision Camera. The HuskyLens integrates an onboard processor capable of performing real time image analysis and color recognition without requiring complex external computation. This makes it particularly suitable for embedded systems where processing resources are limited.

Although the experiment brief initially included color detection using a conventional color sensor, this approach was not implemented due to hardware constraints. Instead, the experiment proceeded with the HuskyLens based method, ensuring that the intended learning outcomes related to image and video input interfacing with a microcontroller were achieved. To enhance system interactivity and provide immediate feedback, an RGB LED module was incorporated to indicate the detected color ID in real time.

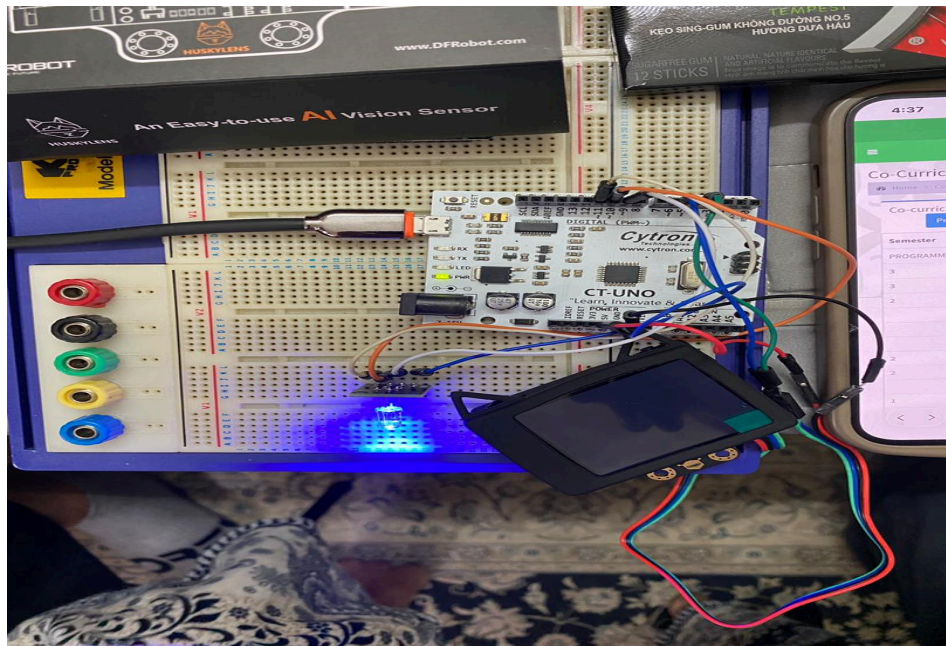
This experiment aims to demonstrate the practical implementation of an AI based color detection system, evaluate its performance, and highlight its advantages in terms of accuracy, consistency, and ease of integration when compared conceptually to traditional sensor based approaches.

EXPERIMENT 9B : Color Detection Using Gravity HuskyLens AI Vision Camera

2.0 MATERIALS AND EQUIPMENTS

1. Arduino Uno microcontroller board
2. Gravity HuskyLens AI Vision Camera
3. RGB LED module
4. Resistors for current limiting of the RGB LED
5. Breadboard
6. Jumper wires
7. USB cable for Arduino power and programming
8. Computer with Arduino IDE installed

3.0 EXPERIMENTAL SETUP



The experimental setup was based on Experiment 9: Color Detection Using Gravity HuskyLens AI Vision Camera, as outlined in the Week 9 Image and Video Input Interfacing laboratory module. Due to the unavailability of the color sensor, the experiment focused exclusively on vision based color detection using the HuskyLens AI Camera.

An Arduino Uno was used as the main microcontroller to interface with the HuskyLens AI Vision Camera. The camera was connected to the Arduino using UART communication, implemented through the SoftwareSerial library since the Arduino Uno has only one hardware serial port. The HuskyLens was powered directly from the Arduino's 5 V supply.

The HuskyLens AI Vision Camera was configured in Color Recognition mode. Using the built in learning function, three different colored objects were trained and assigned unique identification

numbers. Once trained, the camera continuously detected colored objects and transmitted the detected color ID and object position data to the Arduino in real time.

An RGB LED module was connected to the Arduino digital output pins through current limiting resistors. The RGB LED served as a visual indicator to represent the detected color ID. When the HuskyLens detected a trained colored object, the Arduino activated the corresponding color channel of the RGB LED. For example, detection of Color ID 1 activated the red channel of the RGB LED, while other channels remained off.

The system was powered via a USB connection from a computer, which was also used to upload the Arduino program and monitor detection results through the Serial Monitor. This setup enabled real time vision based color detection with immediate visual feedback, fulfilling the learning objectives of image and video input interfacing with a microcontroller.

4.0 METHODOLOGY

The methodology describes the systematic steps used to implement the color detection system using the Gravity HuskyLens AI Vision Camera interfaced with an Arduino Uno. The methodology is divided into circuit assembly, programming logic, and control flow to ensure clear understanding of the system operation.

4.1 Circuit Assembly

The Gravity HuskyLens AI Vision Camera was interfaced with the Arduino Uno using UART communication. Since the Arduino Uno has only one hardware serial port, the **SoftwareSerial** library was used to establish serial communication between the Arduino and the HuskyLens.

The connections were made as follows:

- HuskyLens VCC connected to Arduino 5 V
- HuskyLens GND connected to Arduino GND
- HuskyLens TX connected to Arduino digital pin D4 (SoftwareSerial RX)
- HuskyLens RX connected to Arduino digital pin D5 (SoftwareSerial TX)

An RGB LED module was connected to the Arduino Uno to serve as a visual indicator for detected color IDs. Each color channel of the RGB LED was connected to a separate digital output pin through current limiting resistors. The common pin of the RGB LED was connected to ground.

The complete circuit allowed the Arduino to receive color detection data from the HuskyLens and activate the appropriate RGB LED channel based on the detected color ID.

4.2 Programming Logic

The Arduino program was developed using the Arduino IDE and the DFRobot HuskyLens library. The program logic followed a structured sequence to ensure reliable communication and correct output behavior.

First, the required libraries for HuskyLens communication and SoftwareSerial were included. Serial communication was initialized for debugging via the USB Serial Monitor, while SoftwareSerial was initialized for communication with the HuskyLens camera.

During system initialization, the Arduino continuously attempted to establish communication with the HuskyLens. Once the connection was successful, a confirmation message was displayed on the Serial Monitor.

In the main program loop, the Arduino repeatedly requested detection data from the HuskyLens. When a detected object was available, the camera returned the corresponding color ID and object position data. The Arduino read the detected color ID and compared it with predefined values.

Based on the detected color ID, the Arduino activated the corresponding RGB LED channel. For example, when Color ID 1 was detected, the red channel of the RGB LED was turned on. If Color ID 2 or Color ID 3 was detected, the green or blue channel was activated respectively. All other channels were turned off to ensure that only one color indication was displayed at a time.

4.3 Control Algorithm

The control algorithm of the system follows a continuous sensing–processing–output cycle.

1. System Initialization

- Initialize USB serial communication for monitoring
- Initialize SoftwareSerial for HuskyLens communication
- Establish connection with the HuskyLens AI Vision Camera

2. Color Detection Request

- Continuously request color recognition data from the HuskyLens

3. Data Acquisition

- Check if detection data is available
- Read detected color ID and object information

4. Decision Making

- Compare detected color ID with predefined ID values
- Determine which RGB LED channel should be activated

5. Output Actuation

- Activate the corresponding RGB LED color
- Deactivate other LED channels

6. Loop Execution

- Repeat the detection and output cycle in real time

This control flow enables continuous real time color detection with immediate visual feedback using the RGB LED module.

4.4 Code Used

Arduino

```
#include <SoftwareSerial.h>

#include <DFRobot_HuskyLens.h>

SoftwareSerial mySerial(4, 5); // RX, TX

HUSKYLENS huskylens;

// RGB LED pin definitions

const int redPin = 9;

const int greenPin = 10;

const int bluePin = 11;

void setup() {
  Serial.begin(9600);
  mySerial.begin(9600);

  pinMode(redPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(bluePin, OUTPUT);

  // Turn off all colors initially
  digitalWrite(redPin, LOW);
  digitalWrite(greenPin, LOW);
```

```
digitalWrite(bluePin, LOW);
```

```
while (!huskylens.begin(mySerial)) {  
    Serial.println("HuskyLens not connected!");  
    delay(500);  
}
```

```
Serial.println("HuskyLens Ready");  
}
```

```
void loop() {  
    if (!huskylens.request()) {  
        Serial.println("Request failed");  
        return;  
    }
```

```
    if (huskylens.available()) {  
        HUSKYLENSResult result = huskylens.read();
```

```
        Serial.print("ID: ");  
        Serial.print(result.ID);  
        Serial.print(" X: ");  
        Serial.print(result.xCenter);  
        Serial.print(" Y: ");  
        Serial.println(result.yCenter);
```

```

// Turn off all LEDs first

digitalWrite(redPin, LOW);

digitalWrite(greenPin, LOW);

digitalWrite(bluePin, LOW);


// Light up based on detected ID
if (result.ID == 1) {
    digitalWrite(redPin, HIGH);    // RED ON
    Serial.println("→ Showing RED");
}
else if (result.ID == 2) {
    digitalWrite(greenPin, HIGH);  // GREEN ON
    Serial.println("→ Showing GREEN");
}
else if (result.ID == 3) {
    digitalWrite(bluePin, HIGH);   // BLUE ON
    Serial.println("→ Showing BLUE");
}


delay(1000);

}

}

```

5.0 DATA COLLECTION

OBJECT COLOUR	DETECTED COLOUR ID	COORDINATE X	COORDINATE Y	RGB LED OUTPUT
RED	ID 1	124	11	RED LED ON
GREEN	ID 2	155	57	GREEN LED ON
BLUE	ID 3	113	29	BLUE LED ON

6.0 DATA ANALYSIS

The data collected demonstrates the performance of the color detection system using the Gravity HuskyLens AI Vision Camera interfaced with an Arduino Uno. Three different colored objects were tested, and the detected color ID, object coordinates, and RGB LED output were recorded.

From the results, the HuskyLens successfully identified all three tested colors and assigned the correct color identification numbers. The red object was consistently detected as Color ID 1, the green object as Color ID 2, and the blue object as Color ID 3. This indicates that the color recognition training process was effective and that the camera was able to distinguish between different colors reliably.

In addition to color identification, the HuskyLens provided object position data in the form of X and Y coordinates. The variation in coordinate values reflects the different positions of the objects within the camera's field of view during testing. The system was able to detect objects at different locations without affecting the accuracy of color recognition, demonstrating robustness in spatial detection.

The RGB LED output matched the detected color ID in all test cases. When a red object was detected, the red channel of the RGB LED was activated. Similarly, detection of green and blue objects activated the corresponding green and blue LED channels. This confirms correct data transmission from the HuskyLens to the Arduino and accurate decision making by the control logic.

Overall, the data analysis shows that the system achieved accurate color detection with consistent LED response. The integration of positional data and visual LED output indicates that the system is suitable for real time color based identification applications, such as sorting or visual indication systems.

7.0 RESULT

The color detection system using the Gravity HuskyLens AI Vision Camera operated successfully throughout the experiment. All three trained colored objects were correctly detected by the HuskyLens and assigned their respective color identification numbers.

The system consistently detected the red object as Color ID 1, the green object as Color ID 2, and the blue object as Color ID 3. For each detected object, the HuskyLens provided stable X and Y coordinate values, indicating reliable object localization within the camera's field of view.

The RGB LED module responded correctly to each detected color ID. When Color ID 1 was detected, the red channel of the RGB LED was activated. Detection of Color ID 2 and Color ID 3 activated the green and blue LED channels respectively. At no point were incorrect LED outputs observed during testing.

The results confirm that the communication between the HuskyLens AI Vision Camera and the Arduino Uno was reliable, and the control logic correctly translated detected color IDs into corresponding LED outputs. The system demonstrated stable real time performance under normal indoor lighting conditions.

8.0 DISCUSSION

The color detection system using the Arduino Uno and HuskyLens AI Vision Camera successfully identified three trained colors: red (ID 01), green (ID 02), and blue (ID 03). During testing, the HuskyLens consistently displayed the correct ID and bounding box when a trained colored object was placed within the camera's field of view. The Serial Monitor output confirmed that the detected color IDs corresponded accurately to the expected colors, along with their respective X and Y coordinates. This demonstrates that the HuskyLens color recognition feature is effective for basic object identification and positional tracking, making it suitable for applications such as robotic sorting systems or color-based automation.

Despite the overall successful detection, some discrepancies between the expected and observed outcomes were noted. In certain situations, the camera failed to detect a color immediately, particularly when the object was positioned too far from the lens or when lighting conditions changed. Additionally, when red, green, and blue objects were placed close together, the HuskyLens occasionally detected only one object at a time, even though multiple objects were present. These observations indicate that while the system performs well under controlled conditions, detection accuracy may decrease in more complex or crowded visual environments.

Several limitations and sources of error were identified throughout the experiment. Ambient lighting had a significant impact on color recognition, as shadows or reflections sometimes altered the perceived color. The reliance on prior color calibration also means that inconsistent training can lead to misclassification. Furthermore, the use of SoftwareSerial on the Arduino Uno may introduce communication delays, limiting real-time responsiveness. Overall, the experiment highlights that although the system is reliable for simple color detection tasks, improvements such as consistent lighting, careful calibration, and enhanced hardware could further improve accuracy and system performance.

TASK

Identified Challenges and Improvements :

One of the main challenges encountered during the experiment was sensor calibration, particularly in maintaining consistent color recognition under varying lighting conditions. Changes in ambient light intensity and shadows affected how the HuskyLens perceived red (ID 01), green (ID 02), and blue (ID 03), sometimes leading to delayed or missed detections. Another challenge was the limited field of view and resolution, which reduced detection accuracy when objects were placed too far away or partially occluded. Additionally, the use of SoftwareSerial on the Arduino Uno introduced minor communication delays, which affected real-time responsiveness.

To address these challenges, several improvements can be implemented. First, better calibration techniques such as retraining colors under controlled and consistent lighting can significantly improve detection reliability. Using a fixed light source (e.g., LED ring or light box) can minimize lighting variation.

Insights for Future Iterations :

For future iterations, more advanced algorithms and system configurations can be explored to enhance color detection accuracy and speed. Implementing region-based filtering or tracking algorithms can help the system handle multiple objects simultaneously. Using more powerful microcontrollers (such as Arduino Mega, ESP32, or Raspberry Pi) would allow faster data processing and more stable serial communication.

9.0 CONCLUSION

This experiment successfully demonstrated that the HuskyLens AI Vision Camera integrated with an Arduino Uno can accurately detect and differentiate three trained colors—red (ID 01), green (ID 02), and blue (ID 03) and provide reliable positional data when proper calibration and consistent lighting are applied. The results supported the initial hypothesis that the system is capable of effective multi-color recognition for basic embedded vision applications, despite minor limitations such as sensitivity to lighting variations and occasional detection delays. Overall, the findings highlight the significance of AI-based vision sensors in embedded systems and suggest strong potential for broader applications including robotics, automated sorting, and intelligent control systems, while also providing a solid foundation for future enhancements and more advanced computer vision implementations.

10.0.RECOMMENDATIONS

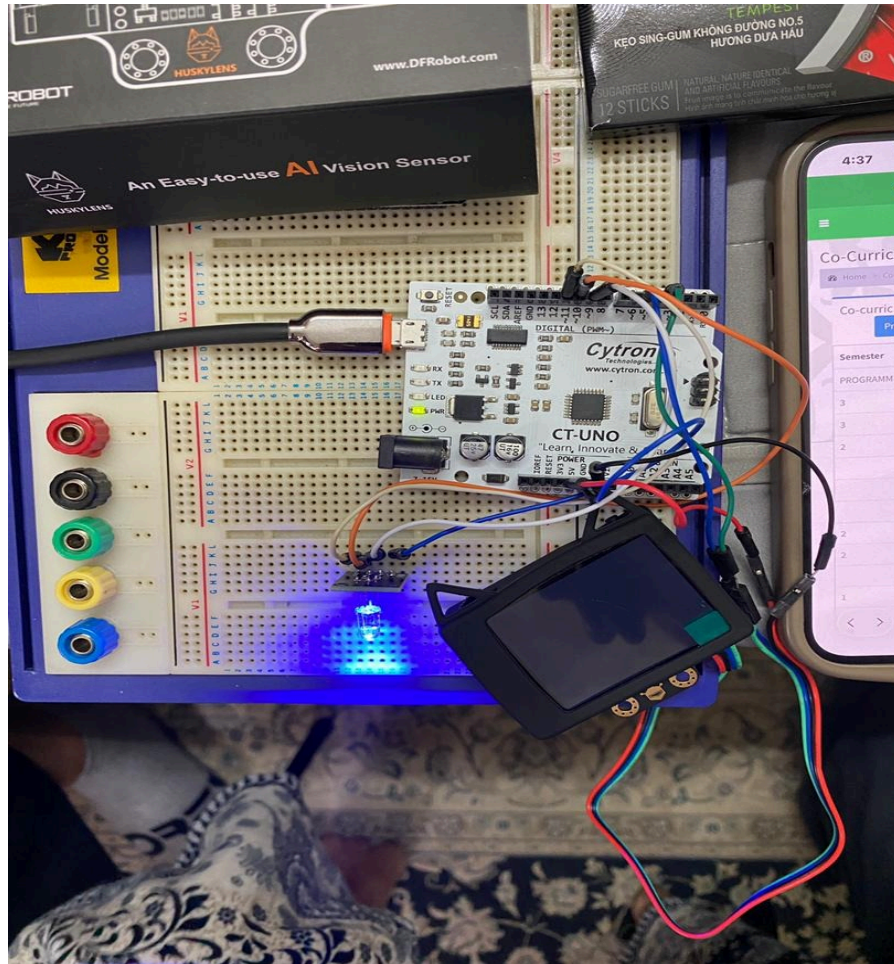
For future iterations of this experiment, several improvements can be made to enhance system performance and reliability. One key recommendation is to conduct color calibration in a controlled lighting environment to minimize the effects of shadows and reflections. Using a fixed and uniform light source, such as LED illumination, can significantly improve color detection accuracy. Additionally, upgrading to a more powerful microcontroller like the ESP32 or Arduino Mega would allow faster data processing and more stable serial communication, reducing delays experienced when using SoftwareSerial on the Arduino Uno.

Another recommended improvement is to expand the system functionality by integrating additional sensors or features. For example, combining the HuskyLens color detection with distance or motion sensors can help validate detected objects and reduce false detections. Future students may also explore advanced HuskyLens modes or simple machine learning techniques to improve object recognition in more complex environments. An important lesson learned from this experiment is the importance of consistent calibration and systematic testing. Careful setup, controlled conditions, and step-by-step validation can greatly improve experimental outcomes and provide valuable experience for students working with AI-based vision systems.

11.0 REFERENCES

1. Gravity: HUSKYLENS - An Easy-to-use AI Machine Vision Sensor. (n.d.).
Wiki.dfrobot.com.
https://wiki.dfrobot.com/HUSKYLENS_V1.0_SKU_SEN0305_SEN0336
2. HuskyLens – An AI Camera: Click, Learn, and Play | Vision Sensor | DFRobot Electronics. (n.d.).
[Www.dfrobot.com. https://www.dfrobot.com/product-1922.html](https://www.dfrobot.com/product-1922.html)
3. Dejan. (2016, May 20). Arduino Color Sensing Tutorial - TCS230 TCS3200 Color Sensor. How to Mechatronics.
<https://howtomechatronics.com/tutorials/arduino/arduino-color-sensing-tutorial->

12.0 APPENDICES



(Wiring diagram for Experiment 9B)

13.0 Acknowledgement

We would like to express our sincere gratitude to Dr. Zulkifli Bin Zainal Abidin and Dr. Wahju Sediono for their continuous support and for providing the essential resources and facilities required to carry out this project. We also extend our heartfelt appreciation to everyone who contributed to the success of this lab report through their valuable insights, assistance, and feedback.

14.0 Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report and that no further improvement on the reports is needed from any of the individual contributors to the report.

Signature:



Name: AHMAD FIRDAUS HUSAINI BIN HASAN AL-BANNA
Matric Number: 2315377

Read [/]

Understand [/]

Agree [/]

Signature:



Read [/]

Name: ADIB ZAFFRY BIN MOHD ABDUL RADZI
Matric Number: 2315149

Understand [/]
Agree [/]

Signature:

Read [/]



Name: UMAR FAROUQI BIN ABU HISHAM
Matric Number: 2314995

Understand [/]
Agree [/]